JVMud User Manual

JVMud Project

Version 0.1.0-SNAPSHOT

Table of Contents

Status

Content to be written.

How To Read This Manual

Content to be written.

Terminology

Content to be written.

Getting Oriented

Content to be written.

Overview

Content to be written.

The JVMud Philosophy Of MUD

JVMud is built around eight core concepts:

- Game
- Text
- Interactive
- Multiplayer
- World
- Persistence
- Temporality
- Presence

Together, these concepts define what JVMud means by MUD.

JVMud is not a general-purpose simulation framework, chat server, collaboration platform, generic game engine, or generic MUD framework. It is specifically and by design an LPMud engine: a platform for imagining, creating, operating, and playing LPC-authored game worlds in the tradition of LP MUDs.

Game

The world exists to be played. JVMud assumes rules, mechanics, goals, challenges, progression, or other game-like structures.

Text

Text is the primary medium through which the world is perceived and manipulated. It is a first-class part of the engine's design, not a skin over a visual game.

Interactive

Players can affect the world, and the world can affect players. Actions have consequences.

Multiplayer

Multiple players inhabit the same world simultaneously. The world is shared.

World

The game takes place within a concrete virtual domain. JVMud describes this domain in terms of Places, Links, Entities, and Movement.

Places give the world locality. Links make Places traversable. Entities are the things that exist in that structure. Movement changes where an Entity is present.

Persistence

The world and its state endure independently of any individual player's session. A player may disconnect and later return to a world whose state has continued to exist.

Persistence does not require eternal continuity. Reboots, resets, and selected save/restore behavior can all be compatible with a persistent world.

Temporality

The world can change through time. Events occur, actions have consequences, and state evolves.

Temporality is separate from Persistence: a world can have active time, saved state, restored state, and reset state without treating all of those as the same idea.

Presence

Presence means that a player experiences the game from within the World. The player is not an outside observer manipulating a simulation from above.

In JVMud terms, a player participates through a Persona: an Entity that serves as the player's current point of perception and action. A Persona is located somewhere, and that location determines what the player can perceive and what actions are locally available.

Players are somewhere, not everywhere. Presence creates locality of information, which makes exploration, discovery, concealment, surprise, navigation, and mystery possible.

LPC Compatibility And Engine Vocabulary

JVMud has one mudlib and language target: LPC for LPMud-style worlds. Its compatibility work exists to make JVMud a deeper LPMud platform, not to broaden it into a multi-language MUD framework.

An LPMud, in JVMud's sense, is a game world authored in LPC, compiled into live game objects, and able to be rewritten, recompiled, and reloaded while the game continues running.

That does not mean JVMud adopts every legacy LPMud driver term as engine ontology.

Legacy names such as rooms, objects, heartbeats, applies, call_outs, and master objects may appear in compatibility surfaces, mudlib source, and LPC-facing documentation. The engine-facing model remains JVMud's own model: Game, Text, Interactive, Multiplayer, World, Persistence, Temporality, and Presence, built from Places, Links, Entities, Players, Sessions, and Personas.

Repository Tour

Content to be written.

Installing And Running

Content to be written.

Installing

Content to be written.

Running JVMud

Content to be written.

Connecting With Telnet

Content to be written.

Smoke Tests

Content to be written.

Playing And Testing A World

Content to be written.

Login Flow

Content to be written.

Basic Commands

Content to be written.

Command Routing

Content to be written.

Transcript Regressions

Content to be written.

Operating JVMud

Content to be written.

Admin CLI

Content to be written.

Runtime Model

Content to be written.

Mudlib Boundary

Content to be written.

LPC And Mudlib Authoring

Content to be written.

Supported LPC

JVMud supports LPC as the source language for LPMud-style game worlds. The goal is not to preserve every historical driver quirk. The goal is to provide a clear, practical LPC platform for writing mudlib code that can be compiled, loaded, inspected, reloaded, and hosted by JVMud.

There is no single authoritative LPC. LPC grew through the work of hobbyists, enthusiasts, and mudlib authors building real worlds on real drivers. Over time, different drivers and mudlibs carried the language in different directions. Many LPC features are therefore best understood as dialect features, compatibility conventions, or driver-specific behavior rather than as parts of one universal standard.

When this manual says JVMud supports LPC, it means JVMud supports the LPC language surface described here, plus the compatibility behavior needed to host the mudlibs JVMud is designed to run.

This topic describes the supported language surface at a high level. It is a working authoring guide, not a complete grammar reference.

Source Files And Objects

An LPC source file compiles to a loadable game object. Source files may define:

- fields
- methods
- forward method declarations
- inherited parent objects

JVMud supports ordinary `inherit` declarations with string paths:

```
inherit "/obj/thing";
```

The parser also recognizes `virtual inherit` syntax for compatibility with mudlib source that uses it:

```
virtual inherit "/obj/base";
```

The runtime resolves object paths through the configured mudlib source root and include paths. Path policy is part of the runtime and mudlib configuration, not the LPC language itself.

Types

JVMud supports the LPC types currently needed by the compiler and hosted mudlibs:

- `int`
- `float`
- `string`
- `status`
- `object`
- `mapping`
- `mixed`
- `void`
- array forms such as `int *`, `string *`, and `mixed *`

Use explicit return types, field types, local variable types, and parameter types in JVMud-maintained LPC. Explicit types make compiler errors clearer and avoid importing legacy driver ambiguity into the engine contract.

```
string short() {
    return "a brass lantern";
}

void set_count(int value) {
    count = value;
}
```

Legacy mudlib source may use patterns that JVMud accepts for compatibility, but new JVMud-facing source should prefer explicit signatures.

Declarations And Modifiers

Fields and methods may use the common LPC declaration modifiers that JVMud currently recognizes:

- `public`
- `private`
- `protected`
- `static`
- `nomask`
- `varargs`
- `nosave`
- `deprecated`

The `varargs` modifier applies to methods only.

```
private int count;
nosave mixed transient_state;

public string query_name() {
    return "lantern";
}

varargs void message(string text, object target) {
    write(text);
}
```

Multiple field declarators are supported in a single declaration:

```
int x, y, z;
string *names, title_words;
```

Statements

JVMud supports the statement forms used by ordinary mudlib control flow:

- blocks
- expression statements
- `if` / `else`
- `while`
- `do` / `while`
- `for`
- `foreach`
- `break`
- `continue`
- `return`

Both `foreach (… in …)` and `foreach (… : …)` forms are supported. Mappings may be iterated with key and value variables.

```
int total(int *values) {
    int sum;

    foreach (int value in values) {
        sum += value;
    }

    return sum;
```

```
}

void describe_mapping(mapping values) {
    foreach (string key, mixed value in values) {
        write(key + ": " + value + "\n");
    }
}
```

Typed local declarations are supported in blocks and in `for` initializers. A `for` initializer may declare one typed local variable.

Expressions

JVMud supports the expression forms expected in practical LPC mudlib code:

- integer, float, string, true, false, and null-like zero values
- array literals
- mapping literals
- local and field access
- assignment and compound assignment
- method calls
- efun calls
- object-style invocation with `->`
- parent calls through `super`
- indexing and slicing
- ternary expressions
- comma expressions in loop clauses
- protected evaluation with `catch`

Supported operator families include arithmetic, comparison, equality, logical, bitwise, shift, and unary operators.

```
mixed *values = ({ "north", "south", "east" });
mapping exits = ([ "north": "/room/green", "south": "/room/road" ]);

string first = values[0];
mixed *rest = values[1..2];

if (sizeof(values) > 0 && exits["north"]) {
    write("There is a way north.\n");
}
```

Adjacent string literals are accepted as one string value, which is useful for long room descriptions:

```
string long() {
    return "The corridor is narrow and cold. "
           "A brass lantern hangs from a hook.";
}
```

Collections

Arrays and mappings are first-class LPC values in JVMud.

Array literals use LPC's `({ ⋯ })` form. Mapping literals use LPC's `([ key: value ])` form.

```
mixed *items = ({ "lamp", "key", "coin" });
mapping descriptions = ([
    "lamp": "A small brass lantern.",
    "key": "A tarnished iron key."
]);
```

Indexing is supported for arrays, mappings, and strings where the runtime type allows it. Slicing is supported for strings and array-like values.

Efun Calls

JVMud exposes engine functions as efuns. Ordinary efun calls use the familiar LPC call form:

```
write("Hello.\n");
sizeof(({ 1, 2, 3 }));
```

Qualified `efun::name(⋯)` calls are supported for engine efuns:

```
return efun::sizeof(({ 1, 2, 3 }));
```

The efun catalog is documented separately in this manual and in the generated Java API documentation for the engine efun registry.

Function References And Inline Callables

JVMud recognizes LPC function-reference forms used by mudlibs, including quoted function references and efun references:

```
#'db_close
efun::db_handles()
```

Inline callable forms used with collection helpers such as `filter` and `map` are supported for the compatibility patterns currently exercised by hosted mudlibs.

Preprocessor And Includes

JVMud includes a preprocessor with support for ordinary include resolution, macro expansion, conditional directives, and source mapping. Include behavior depends on the runtime/compiler configuration for the mudlib being compiled.

Use includes for shared LPC constants and declarations, but keep engine-facing concepts in JVMud vocabulary. LPC headers should help mudlib code compile; they should not redefine the engine's model of Places, Entities, Players, Sessions, or Personas.

Runtime-Oriented LPC

JVMud supports the LPC patterns needed to host live LPMud objects:

- object loading
- method invocation
- inherited behavior
- selected lifecycle methods
- command hooks such as `init`, `add_action`, and `add_verb`
- player/session text output helpers
- save and restore helpers for LPC object state
- timed behavior through scheduler-backed compatibility functions

These features are compatibility surfaces over JVMud's engine model. They make LPC mudlib code usable without making legacy driver vocabulary the engine's own ontology.

Current Limits

JVMud's LPC support is intentionally practical and evolving. If a historical LPC feature is not needed by the hosted mudlibs, it may be absent, partial, or accepted only as a compatibility parse form.

In particular:

- supported LPC means the JVMud compiler and runtime currently have behavior for the construct, not that every legacy driver variant behaves identically
- compatibility syntax may exist before full runtime behavior exists
- JVMud-authored source should prefer explicit types and simple declarations
- engine concepts should be documented as JVMud concepts, not as legacy LPC driver concepts

When in doubt, prefer the forms already used by JVMud mudlib code and covered by compiler or telnet smoke tests.

Explicit Types

Content to be written.

Efuns

Content to be written.

Lifecycle Hooks

Content to be written.

Compatibility Shims

Content to be written.

Mudlibs

Content to be written.

Layout

Content to be written.

LP245 Notes

Content to be written.

Upstream Source Policy

Content to be written.

Developer Reference

Content to be written.

Compiler Pipeline

Content to be written.

Runtime And Server Internals

Content to be written.

Testing And Verification

Content to be written.

Troubleshooting

Content to be written.

Build Failures

Content to be written.

Object Load Failures

Content to be written.

Login And Command Failures

Content to be written.

Documentation Generation

Content to be written.

Appendixes

Content to be written.

Commands

Content to be written.

Configuration Reference

Content to be written.

Compatibility Notes

Content to be written.

Glossary

Content to be written.